

Video Plan for uVisionEmbeddedStateGraphTimer and RustEmbeddedStateVertex Projects

stevemac321

1 Introduction

This video will cover both the C-based `uVisionEmbeddedStateGraphTimer` project and the Rust-based `RustEmbeddedStateVertex` project. It is designed to appeal to embedded systems developers using either language. The agenda will include timestamps in the YouTube description so viewers can jump to the sections they are most interested in.

2 Agenda and Section Order

Here is the planned order of the video:

2.1 (a) A Review of uVision

- An overview of the Keil uVision IDE and its use for embedded development with STM32 microcontrollers.
- Introduction to the STM32F401RE microcontroller and its relevance in embedded systems development.

2.2 (b) Step Through the Startup Code to `main()`

- Explanation of the startup code and memory map.
- Description of how the linker file represents the binary image.
- How data is loaded into RAM and how instructions are executed from flash.
- Introduction to the PC register as the "pre-fetch" mechanism and the multi-stage pipeline in the STM32.
- Emphasize that this is a bare-metal environment with no operating system.

2.3 (c) Demo of the C Program

- Show the C program running in uVision.
- Explain how the program polls for system state and builds the graph structure.
- Discussion of the vertex and edge structure using bitvectors, and how interrupts handle real-time events.
- Highlight the use of atomics for thread-safe access to shared data between the main loop and interrupts.
- Demonstrate the Breadth-First Search (BFS) algorithm in action.

2.4 (d) Simplified Rust Version Demo in VSCode on Linux

- Transition to the Rust version, showing its setup in VSCode on Linux.
- Explanation of how Rust can be used for embedded development, with comparisons to the C version.
- Highlight the similarities and differences between the C and Rust implementations.

3 Conclusion

- Recap the key topics covered, including embedded development in both C and Rust, memory handling, and concurrency with atomics and interrupts.
- Remind viewers to check the YouTube description for timestamps to easily navigate through the video.

4 Next Steps

- Once the Segger J-Link is working and a print dump of the graph is added, the recording can be done.
- The current plan will serve as a guide for organizing the video, but there is room for improvement and further planning, which can be worked on later.